AWS VPC Peering Project

Project Title: Connecting Two Amazon VPCs Using VPC Peering

Name: Hiral Jagetiya

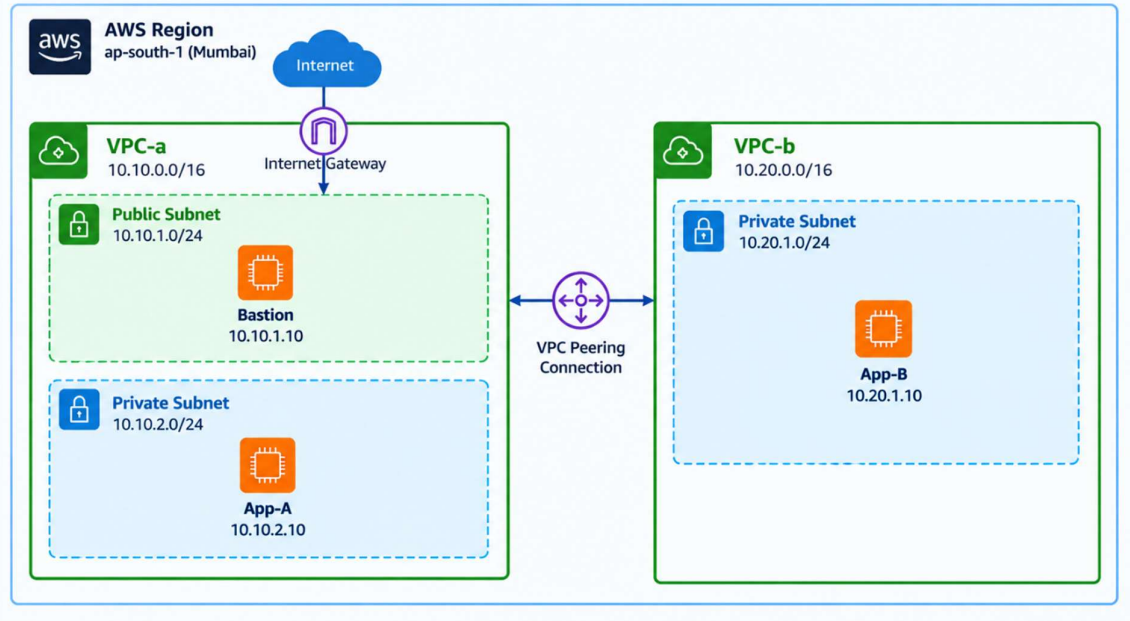
AWS Region: Asia Pacific (Mumbai) — ap-south-1

Completion Date: 28 October 2026

Project Objective

The objective of this project is to establish private network connectivity between two Amazon Virtual Private Clouds (VPCs) using VPC Peering. The project demonstrates VPC creation, subnet configuration, routing, security group configuration, and connectivity testing between EC2 instances.

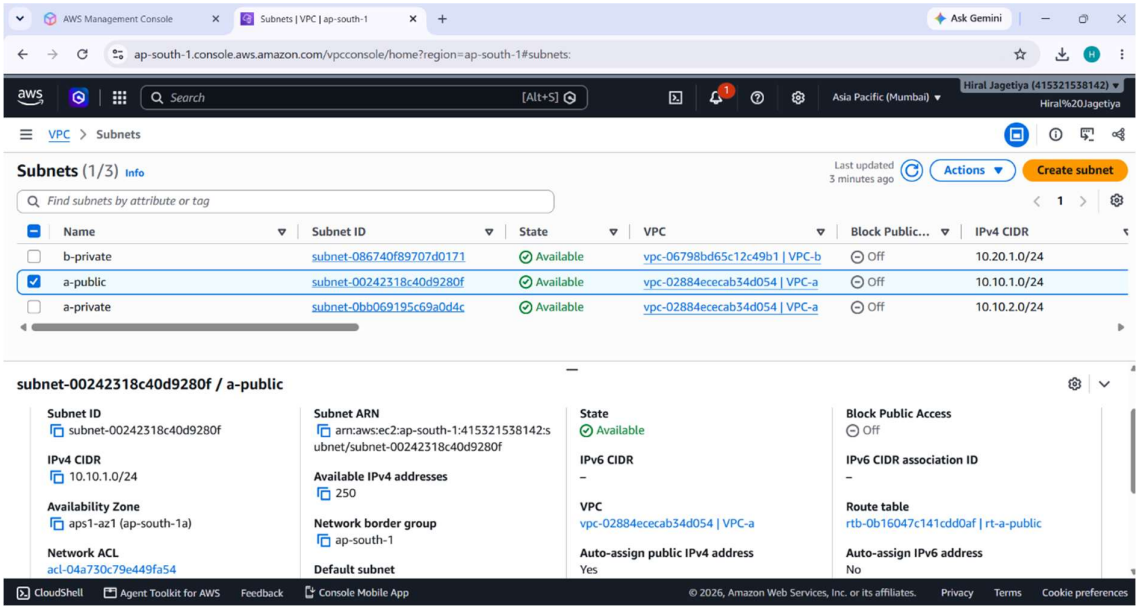
Architecture diagram



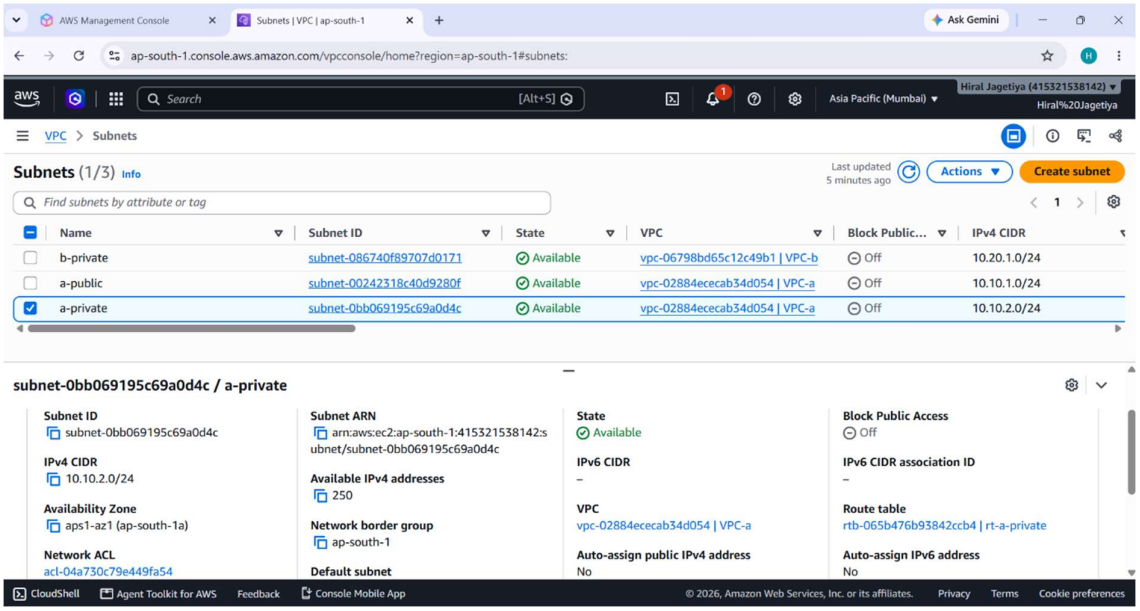
AWS Resource Inventory

Resource Category	Resource Name / Identifier	VPC / Associated Context	Key Details & Configurations
EC2 Instance	bastion	VPC-a	Running, t3.micro, Public IP 52.66.124.124, Private IP 10.10.1.10
EC2 Instance	app-a	VPC-a	Running, t3.micro, Private Subnet a-private (10.10.2.0/24)
EC2 Instance	app-b	VPC-b	Running, t3.micro, Private Subnet b-private (10.20.1.0/24)
Subnet	a-public	VPC-a	CIDR: 10.10.1.0/24, Auto-assign Public IPv4: Yes
Subnet	a-private	VPC-a	CIDR: 10.10.2.0/24, Auto-assign Public IPv4: No
Subnet	b-private	VPC-b	CIDR: 10.20.1.0/24, Auto-assign Public IPv4: No

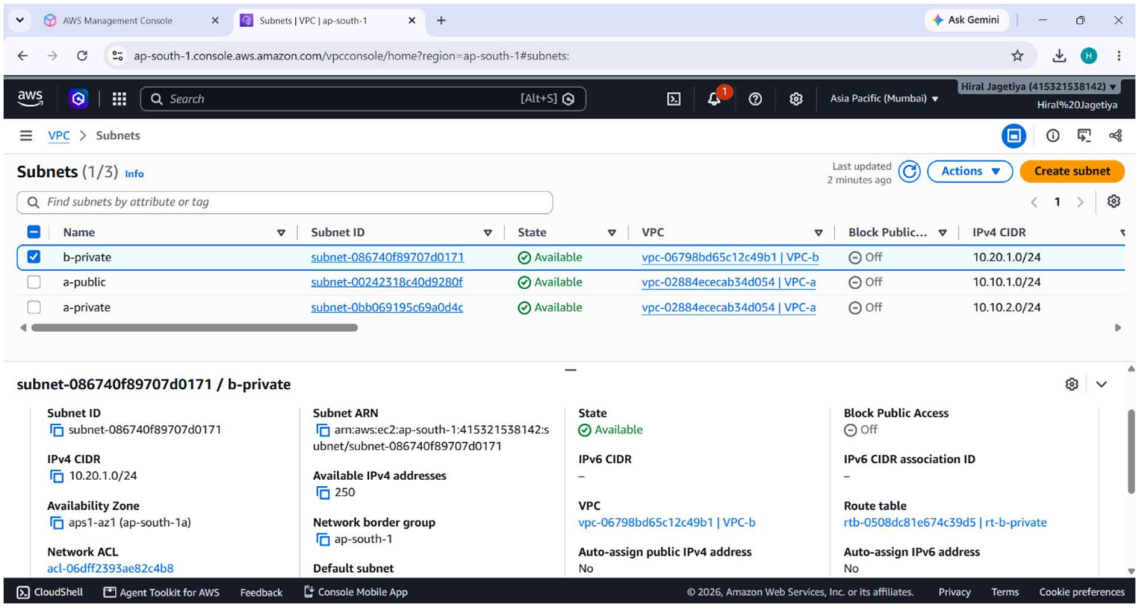
Route Table	rt-a-public	VPC-a	Associated with public subnet a-public
Route Table	rt-a-private	VPC-a	Associated with private subnet a-private
Route Table	rt-b-private	VPC-b	Associated with private subnet b-private
Internet Gateway	igw-a	VPC-a	State: Attached
VPC Peering Connection	Inter-VPC Peering	VPC-a & VPC-b	Routes traffic between VPC-a (10.10.0.0/16) and VPC-b (10.20.0.0/16)
Security Group	bastion-sg	VPC-a	Inbound Rule: SSH (TCP Port 22)
Security Group	app-a-sg	VPC-a	Inbound Rules: SSH (TCP 22), All ICMP - IPv4
Security Group	app-b-sg	VPC-b	Inbound Rules: SSH (TCP 22); All ICMP-IPv4



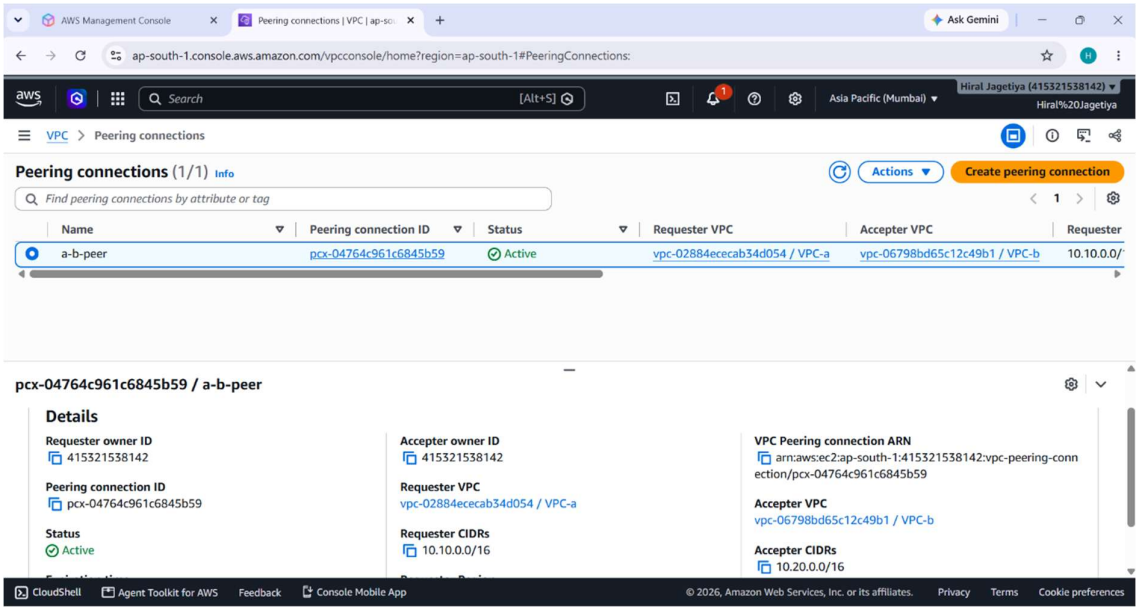
Public subnet -A



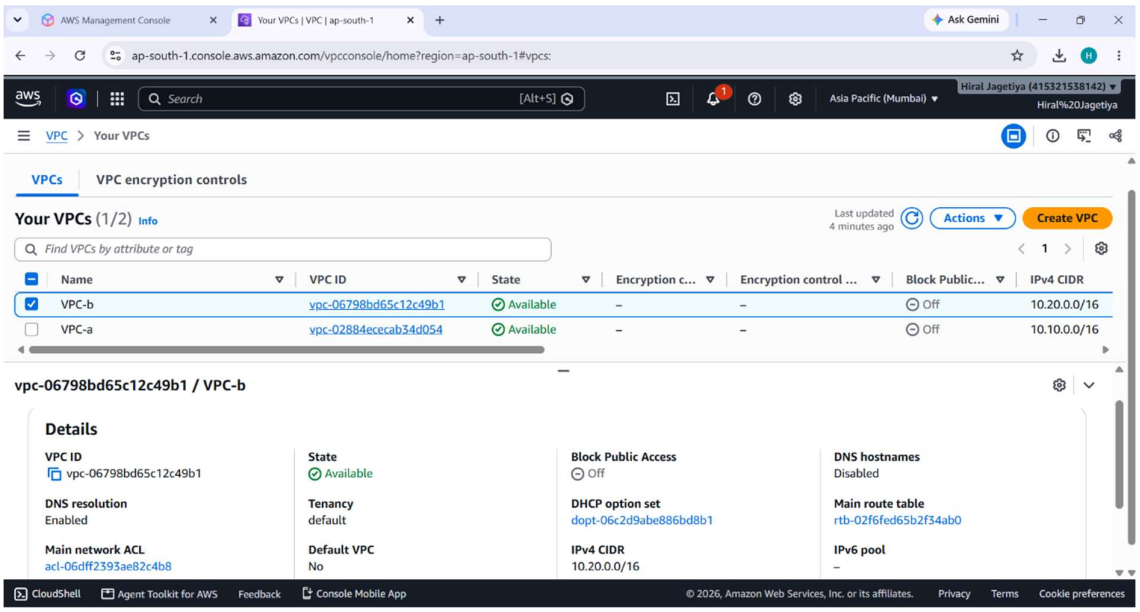
Private Subnet -A



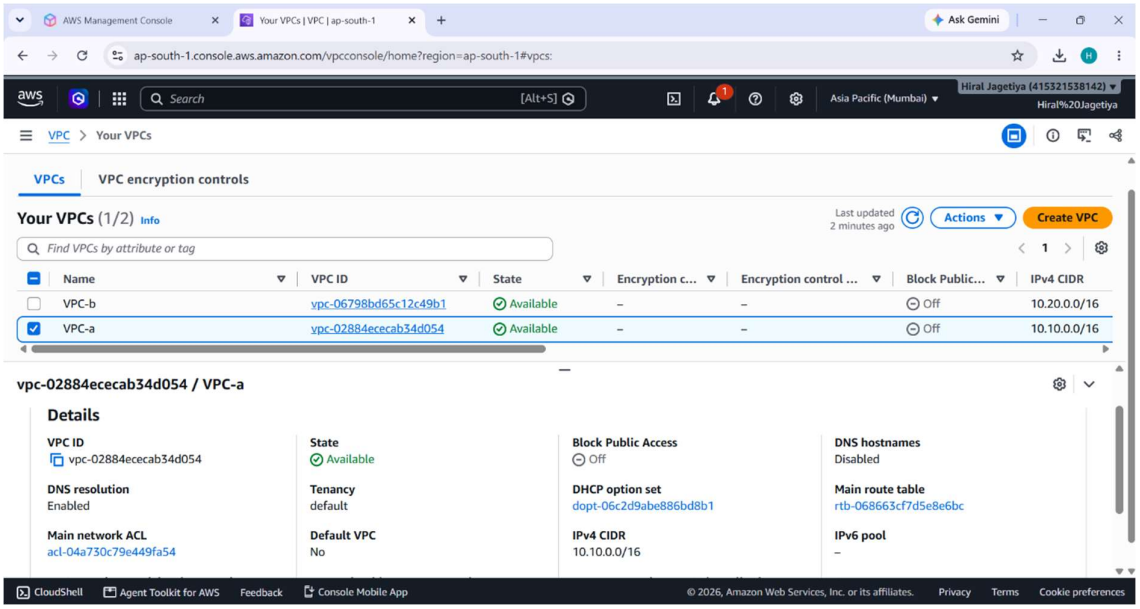
Private Subnet -B



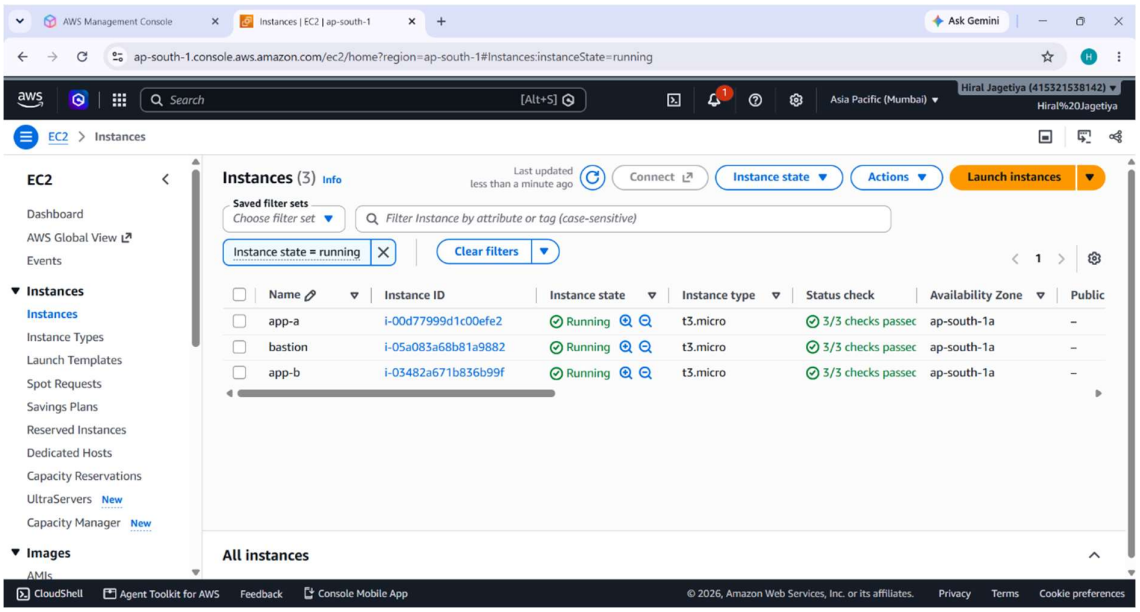
Active VPC Peering Connection



VPC - B



VPC – A



EC2 Instances

```
ec2-user@ip-10-10-1-10:~
    inet 10.10.1.10/24 metric 512 brd 10.10.1.255 scope global dynamic ens5
       valid_lft 3553sec preferred_lft 3553sec
[ec2-user@ip-10-10-1-10 ~]$ ssh ec2-user@10.10.2.10
The authenticity of host '10.10.2.10 (10.10.2.10)' can't be established.
ED25519 key fingerprint is SHA256:ljTO80nGs1uklqFcDj+UOChmbyRd8WMwpm42R+8JyM4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? no
Host key verification failed.
[ec2-user@ip-10-10-1-10 ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.166 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.275 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.175 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.178 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3089ms
rtt min/avg/max/mdev = 0.166/0.198/0.275/0.044 ms
[ec2-user@ip-10-10-1-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3156ms

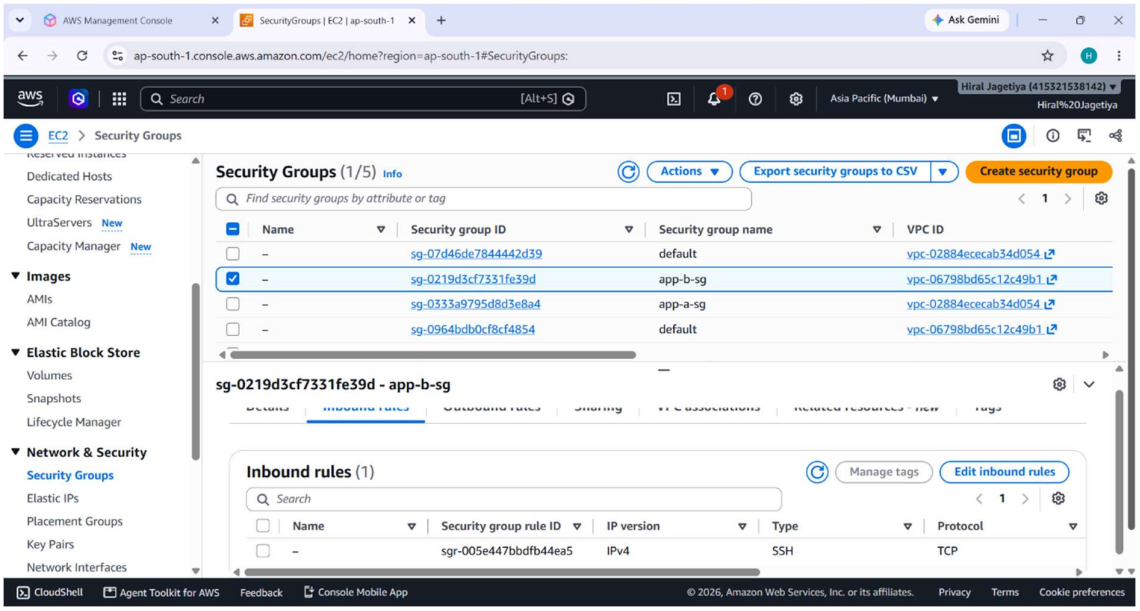
[ec2-user@ip-10-10-1-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3086ms

[ec2-user@ip-10-10-1-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.198 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.201 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.191 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.212 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3102ms
rtt min/avg/max/mdev = 0.191/0.200/0.212/0.007 ms
[ec2-user@ip-10-10-1-10 ~]$
```

Successful Ping Test



B-ICMP Removed

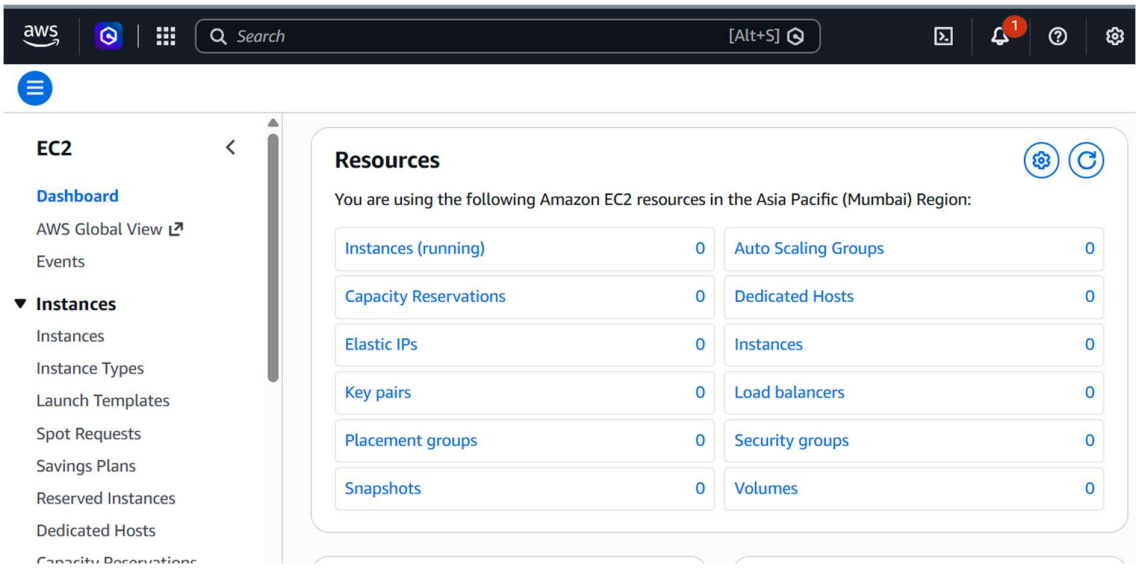
```
[ec2-user@ip-10-10-1-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.198 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.201 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.191 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.212 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3102ms
rtt min/avg/max/mdev = 0.191/0.200/0.212/0.007 ms
[ec2-user@ip-10-10-1-10 ~]$
```

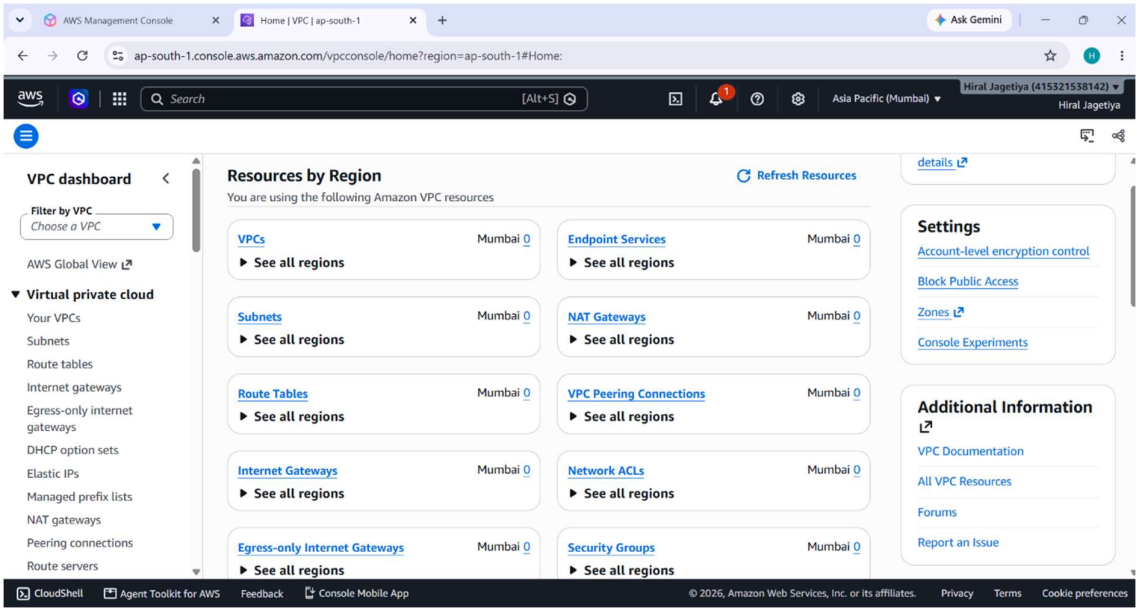
Before removing ICMP

Controlled ICMP Failure and Recovery

Controlled ICMP Failure and Recovery means intentionally blocking ping traffic, confirming that ping fails, then restoring the rule and confirming that ping works again.



Clean up Evidence (EC2)



Clean Up Evidence (VPC)

Procedure

1. **Create VPCs:** Create VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16) in Mumbai region (ap-south-1).
2. **Create Subnets:** Create three subnets:

   VPC A Public: 10.10.1.0/24

   VPC A Private: 10.10.2.0/24

   VPC B Private: 10.20.1.0/24

3. **Create Internet Gateway:** Create ewb-igw-a and attach it to VPC A.
4. **Configure Route Tables:** Create three route tables and associate them with their respective subnets. Add the Internet Gateway route to the public route table.
5. **Create VPC Peering:** Create ewb-a-b-peer between VPC A and VPC B and accept the request.
6. **Add Peering Routes:** Add routes to both VPCs so they can communicate through the peering connection.
7. **Configure Security Groups:** Allow required SSH and ICMP traffic between the instances.
8. **Launch EC2 Instances:** Create three instances:

   Bastion: 10.10.1.10

   App-A: 10.10.2.10

   App-B: 10.20.1.10

9. **Connect to Bastion:** Connect to the Bastion instance using SSH and your private key.
10. **Test Connectivity:** Use ping to test communication between Bastion, App-A, and App-B. Test SSH access to App-B if required.
11. **Test ICMP Failure and Recovery:** Temporarily remove the relevant ICMP rule, verify the ping failure, restore the rule, and test again.

Learning Outcomes

1. Learned to create and configure VPCs, subnets, and route tables in AWS.
2. Understood the purpose and configuration of VPC Peering.
3. Learned to establish private communication between two VPCs.
4. Gained knowledge of routing traffic through route tables and an Internet Gateway.
5. Learned to configure Security Groups to control inbound traffic.
6. Practiced launching EC2 instances in public and private subnets.
7. Understood how to test network connectivity using ping and SSH.
8. Learned to troubleshoot connectivity issues by checking routes and Security Group rules.
9. Understood the importance of secure network access and resource cleanup in AWS.